

Object Oriented Programming Complete C++ Program Answer Sheet

Basic I/O · Classes & Objects · Inheritance · Polymorphism · Operator Overloading · Templates · File
Handling · Exception Handling · STL

Each program includes: Code, Sample Output, Dry Run, and Explanation

Contents

1. Hello World
2. Input/Output Using cin, cout
3. Swap two numbers (with temp)
4. Swap two numbers (without temp)
5. Prime Number
6. Factorial (loop)
7. Even or Odd
8. Leap Year Checker
9. Calculator
10. Reverse a number
11. Palindrome
- 12-15. Duplicate Entries
16. Factorial (Recursion)
17. Fibonacci
18. GCD/LCM
19. Class and Object
20. Constructor & Destructor
21. Access Modifiers
22. Encapsulation Example
23. Data Abstraction
24. Inline Functions in Class
25. Static Data Members & Functions
26. Friend Function
27. This Pointer
28. Const Member Functions
29. Single Inheritance
30. Multiple Inheritance
31. Multilevel Inheritance
32. Hierarchical Inheritance
33. Hybrid Inheritance
34. Function Overloading
35. Operator Overloading
36. Function Overriding
37. Virtual Functions
38. Class Templates
39. Template with Multiple Types
40. Write and Read from File
41. Count Lines/Words in File
42. Copy One File to Another
43. Try-Catch Block
44. Custom Exception
45. Nested Exception
46. Vector Operations
47. Map (Key-Value)
48. Stack/Queue from STL
49. Set/Multiset
50. Constant Data Member Program
51. Const Member Function Program

- 52. Static Data Member Program
- 53. Static Member Function Program
- 54. Overloading Binary Operator +
- 55. Overloading Unary Operator -
- 56. Overloading == Operator
- 57. Overloading << and >> Operators
- 58. Virtual Function with Base and Derived Class
- 59. Virtual Function with Pointer to Base Class
- 60. Pure Virtual Function and Abstract Class
- 61. Virtual Destructors (Dynamic Binding in Destruction)
- 62. Exception Handling
- 63. Function and Class Templates
- 64. Stream Classes (cin, cout, ifstream, ofstream)
- 65. Dynamic Memory Allocation (new and delete)
- 66. Concept of Reference Variable
- 66 (dup). Duplicate Entry

Program 1: Hello World

First basic C++ program.

C++ Code:

```
#include <iostream>
using namespace std;

int main()
{
    cout << "Hello, World!" << endl;
    return 0;
}
```

Sample Output:

Hello, World!

Dry Run:

The program has no input. `main()` executes a single `cout` statement that sends the string "Hello, World!" to standard output, followed by a newline (`endl`).

Explanation:

This is the simplest C++ program structure: it includes the `iostream` header for input/output, uses the `std` namespace so `cout` does not need to be written as `std::cout`, and `main()` returns 0 to indicate successful execution.

Program 2: Input/Output Using cin, cout

Using cin, cout.

C++ Code:

```
#include <iostream>
using namespace std;

int main()
{
    int age;
    string name;

    cout << "Enter your name: ";
    cin >> name;
    cout << "Enter your age: ";
    cin >> age;

    cout << "Hello " << name << ", you are " << age << " years old." << endl;
    return 0;
}
```

Sample Output:

```
Enter your name: Tanishque
Enter your age: 20
Hello Tanishque, you are 20 years old.
```

Dry Run:

Input: name = "Tanishque", age = 20. cin >> name reads the first whitespace-delimited token into name; cin >> age reads the next token and converts it to an int. The final cout concatenates these values with literal text using the << operator chain.

Explanation:

cin (console input) extracts formatted data from standard input into variables; cout (console output) inserts data to standard output. The >> and << operators can be chained to read/write multiple values in one statement.

Program 3: Swap two numbers (with temp)

Using a temporary variable.

C++ Code:

```
#include <iostream>
using namespace std;

int main()
{
    int a = 5, b = 10, temp;

    cout << "Before swap: a=" << a << " b=" << b << endl;

    temp = a;
    a = b;
    b = temp;

    cout << "After swap: a=" << a << " b=" << b << endl;
    return 0;
}
```

Sample Output:

```
Before swap: a=5 b=10
After swap: a=10 b=5
```

Dry Run:

a=5, b=10. temp=a -> temp=5. a=b -> a=10. b=temp -> b=5. Final: a=10, b=5.

Explanation:

The classic three-variable swap technique: a temporary variable holds one value while it is overwritten, ensuring no data is lost during the exchange.

Program 4: Swap two numbers (without temp)

Using arithmetic, no temp variable.

C++ Code:

```
#include <iostream>
using namespace std;

int main()
{
    int a = 5, b = 10;

    cout << "Before swap: a=" << a << " b=" << b << endl;

    a = a + b;    // a becomes 15
    b = a - b;    // b becomes 5 (original a)
    a = a - b;    // a becomes 10 (original b)

    cout << "After swap: a=" << a << " b=" << b << endl;
    return 0;
}
```

Sample Output:

```
Before swap: a=5 b=10
After swap: a=10 b=5
```

Dry Run:

a=5, b=10. a=a+b -> a=15. b=a-b -> b=15-10=5. a=a-b -> a=15-5=10. Final: a=10, b=5.

Explanation:

This swaps two numbers using only arithmetic, avoiding a third variable. It works by encoding both original values into the sum stored temporarily in a, then extracting them back out. Note: this can overflow for very large values; XOR swap is an alternative for integers that avoids overflow.

Program 5: Prime Number

Check for prime.

C++ Code:

```
#include <iostream>
using namespace std;

int main()
{
    int n, i;
    bool isPrime = true;

    cout << "Enter a number: ";
    cin >> n;

    if (n <= 1)
        isPrime = false;

    for (i = 2; i * i <= n; i++)
    {
        if (n % i == 0)
        {
            isPrime = false;
            break;
        }
    }

    if (isPrime)
        cout << n << " is a Prime number." << endl;
    else
        cout << n << " is NOT a Prime number." << endl;

    return 0;
}
```

Sample Output:

```
Enter a number: 29
29 is a Prime number.
```

Dry Run:

n=29. Loop checks i=2 (29%2=1), i=3 (29%3=2), i=4 (29%4=1); i*i=25<=29 continues, i=5 -> i*i=25<=29, 29%5=4; i=6 -> i*i=36>29, loop stops. isPrime stays true.

Explanation:

A number is prime if it has no divisors other than 1 and itself. The loop only needs to check divisors up to $\sqrt{n}$ (implemented as $i*i \leq n$), since any factor larger than $\sqrt{n}$ would have a corresponding factor smaller than it, already checked.

Program 6: Factorial (loop)

Using a loop.

C++ Code:

```
#include <iostream>
using namespace std;

int main()
{
    int n;
    unsigned long long fact = 1;

    cout << "Enter a number: ";
    cin >> n;

    for (int i = 1; i <= n; i++)
        fact *= i;

    cout << "Factorial of " << n << " = " << fact << endl;
    return 0;
}
```

Sample Output:

```
Enter a number: 5
Factorial of 5 = 120
```

Dry Run:

n=5. fact starts at 1. i=1: fact=1. i=2: fact=2. i=3: fact=6. i=4: fact=24. i=5: fact=120.

Explanation:

Factorial of n ($n!$) is the product of all positive integers from 1 to n . The loop accumulates this product iteratively. unsigned long long is used to accommodate larger factorial values without overflow for a reasonable range of n .

Program 7: Even or Odd

Check if a number is even or odd.

C++ Code:

```
#include <iostream>
using namespace std;

int main()
{
    int n;

    cout << "Enter a number: ";
    cin >> n;

    if (n % 2 == 0)
        cout << n << " is Even." << endl;
    else
        cout << n << " is Odd." << endl;

    return 0;
}
```

Sample Output:

```
Enter a number: 7
7 is Odd.
```

Dry Run:

n=7. $n \% 2$ evaluates to 1 (non-zero), so the else branch executes, printing "Odd".

Explanation:

The modulus operator % gives the remainder of integer division. Any number divisible evenly by 2 (remainder 0) is even; otherwise it is odd.

Program 8: Leap Year Checker

Check if a year is a leap year.

C++ Code:

```
#include <iostream>
using namespace std;

int main()
{
    int year;

    cout << "Enter a year: ";
    cin >> year;

    if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))
        cout << year << " is a Leap Year." << endl;
    else
        cout << year << " is NOT a Leap Year." << endl;

    return 0;
}
```

Sample Output:

```
Enter a year: 2000
2000 is a Leap Year.
```

Dry Run:

year=2000. year%4==0 is true (2000%4=0). year%100!=0 is false (2000%100=0), so the first AND clause is false. But year%400==0 is true (2000%400=0), so the overall OR is true.

Explanation:

A year is a leap year if it is divisible by 4, except for century years (divisible by 100), which must also be divisible by 400 to qualify. This correctly handles edge cases like 1900 (not a leap year) and 2000 (a leap year).

Program 9: Calculator

Basic arithmetic operations.

C++ Code:

```
#include <iostream>
using namespace std;

int main()
{
    double a, b;
    char op;

    cout << "Enter first number: ";
    cin >> a;
    cout << "Enter operator (+, -, *, /): ";
    cin >> op;
    cout << "Enter second number: ";
    cin >> b;

    switch (op)
    {
        case '+': cout << "Result: " << a + b << endl; break;
        case '-': cout << "Result: " << a - b << endl; break;
        case '*': cout << "Result: " << a * b << endl; break;
        case '/':
            if (b != 0)
                cout << "Result: " << a / b << endl;
            else
                cout << "Error: Division by zero." << endl;
            break;
        default:
            cout << "Invalid operator." << endl;
    }

    return 0;
}
```

Sample Output:

```
Enter first number: 10
Enter operator (+, -, *, /): *
Enter second number: 4
Result: 40
```

Dry Run:

a=10, op='*', b=4. The switch matches case '*', computing a*b = 40 and printing it.

Explanation:

A switch statement selects the correct arithmetic operation based on the operator character entered. Division includes a zero-check to avoid undefined behavior.

Program 10: Reverse a number

Reverse using a loop.

C++ Code:

```
#include <iostream>
using namespace std;

int main()
{
    int n, reversed = 0, digit;

    cout << "Enter a number: ";
    cin >> n;

    int original = n;

    while (n != 0)
    {
        digit = n % 10;
        reversed = reversed * 10 + digit;
        n /= 10;
    }

    cout << "Reversed number of " << original << " = " << reversed << endl;
    return 0;
}
```

Sample Output:

```
Enter a number: 1234
Reversed number of 1234 = 4321
```

Dry Run:

n=1234. Iter1: digit=4, reversed=4, n=123. Iter2: digit=3, reversed=43, n=12. Iter3: digit=2, reversed=432, n=1. Iter4: digit=1, reversed=4321, n=0. Loop ends.

Explanation:

Each iteration extracts the last digit of n using $n \% 10$, appends it to reversed by shifting reversed left one decimal place ($\times 10$) and adding the digit, then removes the last digit from n using integer division by 10.

Program 11: Palindrome

Check numeric/string palindrome.

C++ Code:

```
#include <iostream>
#include <string>
using namespace std;

bool isPalindromeString(string s)
{
    int left = 0, right = s.length() - 1;
    while (left < right)
    {
        if (s[left] != s[right])
            return false;
        left++;
        right--;
    }
    return true;
}

int main()
{
    int n, original, reversed = 0, digit;
    string s;

    cout << "Enter a number: ";
    cin >> n;
    original = n;

    while (n != 0)
    {
        digit = n % 10;
        reversed = reversed * 10 + digit;
        n /= 10;
    }

    if (original == reversed)
        cout << original << " is a numeric Palindrome." << endl;
    else
        cout << original << " is NOT a numeric Palindrome." << endl;

    cout << "Enter a string: ";
    cin >> s;

    if (isPalindromeString(s))
        cout << s << " is a string Palindrome." << endl;
    else
        cout << s << " is NOT a string Palindrome." << endl;

    return 0;
}
```

Sample Output:

```
Enter a number: 121
121 is a numeric Palindrome.
Enter a string: madam
madam is a string Palindrome.
```

Dry Run:

Numeric: n=121 reverses to 121 (matches original -> palindrome). String: s="madam", left=0/right=4 ('m'=='m'), left=1/right=3 ('a'=='a'), left=2/right=2 loop ends -> palindrome.

Explanation:

A palindrome reads the same forwards and backwards. For numbers, this is checked by reversing the digits and comparing to the original. For strings, two pointers move inward from both ends, comparing characters at each step.

Program 12-15: Duplicate Entries

Repeated items from the source list.

Items 12 (Leap Year Checker), 13 (Calculator), 14 (Reverse a number), and 15 (Palindrome) are exact repeats of items 8, 9, 10, and 11 in the original question list (same title and description). See Programs 8, 9, 10, and 11 above respectively.

Program 16: Factorial (Recursion)

Using recursion.

C++ Code:

```
#include <iostream>
using namespace std;

unsigned long long factorial(int n)
{
    if (n == 0 || n == 1)
        return 1;
    return n * factorial(n - 1);
}

int main()
{
    int n;

    cout << "Enter a number: ";
    cin >> n;

    cout << "Factorial of " << n << " = " << factorial(n) << endl;
    return 0;
}
```

Sample Output:

```
Enter a number: 5
Factorial of 5 = 120
```

Dry Run:

$\text{factorial}(5) = 5 * \text{factorial}(4) = 5 * 4 * \text{factorial}(3) = 5 * 4 * 3 * \text{factorial}(2) = 5 * 4 * 3 * 2 * \text{factorial}(1) = 5 * 4 * 3 * 2 * 1 = 120$
(base case $\text{factorial}(1)$ returns 1).

Explanation:

Unlike the loop version, this expresses factorial recursively: $n! = n \times (n-1)!$, with the base case $0! = 1! = 1$ stopping the recursion. Each call waits for the recursive call below it to return before multiplying, building the result on the way back up the call stack.

Program 17: Fibonacci

Recursive and loop versions.

C++ Code:

```
#include <iostream>
using namespace std;

int fibRecursive(int n)
{
    if (n <= 1)
        return n;
    return fibRecursive(n - 1) + fibRecursive(n - 2);
}

int main()
{
    int n;
    cout << "Enter number of terms: ";
    cin >> n;

    cout << "Fibonacci series (loop): ";
    int a = 0, b = 1, next;
    for (int i = 0; i < n; i++)
    {
        cout << a << " ";
        next = a + b;
        a = b;
        b = next;
    }
    cout << endl;

    cout << "Fibonacci series (recursive): ";
    for (int i = 0; i < n; i++)
        cout << fibRecursive(i) << " ";
    cout << endl;

    return 0;
}
```

Sample Output:

```
Enter number of terms: 6
Fibonacci series (loop): 0 1 1 2 3 5
Fibonacci series (recursive): 0 1 1 2 3 5
```

Dry Run:

Loop version: a=0,b=1 -> print 0, next=1,a=1,b=1 -> print 1, next=2,a=1,b=2 -> print 1, next=3,a=2,b=3 -> print 2, ... producing 0 1 1 2 3 5. Recursive version computes each term independently via fibRecursive(i), giving the same sequence.

Explanation:

The Fibonacci sequence is defined as $F(n) = F(n-1) + F(n-2)$, with $F(0)=0$ and $F(1)=1$. The loop version computes it efficiently in $O(n)$ time using two running variables. The recursive version directly mirrors the mathematical definition but recomputes overlapping subproblems, making it exponential ($O(2^n)$) without memoization.

Program 18: GCD/LCM

Using the Euclidean algorithm.

C++ Code:

```
#include <iostream>
using namespace std;

int gcd(int a, int b)
{
    while (b != 0)
    {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

int main()
{
    int a, b;

    cout << "Enter two numbers: ";
    cin >> a >> b;

    int g = gcd(a, b);
    int l = (a * b) / g;

    cout << "GCD = " << g << endl;
    cout << "LCM = " << l << endl;

    return 0;
}
```

Sample Output:

```
Enter two numbers: 12 18
GCD = 6
LCM = 36
```

Dry Run:

a=12, b=18. Iter1: t=18, b=12%18=12, a=18. Iter2: t=12, b=18%12=6, a=12. Iter3: t=6, b=12%6=0, a=6. b==0, loop ends, gcd=6. LCM = (12*18)/6 = 216/6 = 36.

Explanation:

The Euclidean algorithm computes GCD by repeatedly replacing the larger number with the remainder of dividing it by the smaller number, until the remainder is 0. LCM is then derived using the identity $LCM(a,b) = (a \times b) / GCD(a,b)$.

Program 19: Class and Object

Create and use a simple class.

C++ Code:

```
#include <iostream>
using namespace std;

class Student
{
public:
    string name;
    int rollNo;

    void display()
    {
        cout << "Name: " << name << ", Roll No: " << rollNo << endl;
    }
};

int main()
{
    Student s1;
    s1.name = "Tanishque";
    s1.rollNo = 101;

    s1.display();
    return 0;
}
```

Sample Output:

Name: Tanishque, Roll No: 101

Dry Run:

An object s1 of class Student is created. Its data members name and rollNo are set directly using the dot operator, then display() is called on s1 to print them.

Explanation:

A class is a user-defined blueprint that bundles data (data members) and behavior (member functions). An object is a concrete instance of that class, with its own copy of the data members.

Program 20: Constructor & Destructor

Default, parameterized, and copy constructors.

C++ Code:

```
#include <iostream>
using namespace std;

class Box
{
public:
    int length;

    Box()                // default constructor
    {
        length = 0;
        cout << "Default constructor called." << endl;
    }

    Box(int l)           // parameterized constructor
    {
        length = l;
        cout << "Parameterized constructor called, length=" << l << endl;
    }

    Box(const Box &b)     // copy constructor
    {
        length = b.length;
        cout << "Copy constructor called, length=" << length << endl;
    }

    ~Box()               // destructor
    {
        cout << "Destructor called for length=" << length << endl;
    }
};

int main()
{
    Box b1;
    Box b2(10);
    Box b3 = b2;

    return 0;
}
```

Sample Output:

```
Default constructor called.
Parameterized constructor called, length=10
Copy constructor called, length=10
Destructor called for length=10
Destructor called for length=10
Destructor called for length=0
```

Dry Run:

b1 is constructed with the default constructor (length=0). b2 is constructed with the parameterized constructor (length=10). b3 is constructed by copying b2's data using the copy constructor (length=10). At program end, destructors run in reverse order of construction: b3, then b2, then b1.

Explanation:

A constructor initializes an object when it is created; C++ allows multiple constructors (overloading) to support different initialization styles. The copy constructor creates a new object as a copy of an existing one. The

destructor runs automatically when an object goes out of scope, used for cleanup.

Program 21: Access Modifiers

Use of private, public, protected.

C++ Code:

```
#include <iostream>
using namespace std;

class Account
{
private:
    double balance;          // accessible only within the class

protected:
    string accountType;      // accessible within class and derived classes

public:
    string accountNumber;    // accessible from anywhere

    void setBalance(double b) { balance = b; }
    double getBalance() { return balance; }
};

int main()
{
    Account a;
    a.accountNumber = "ACC123";    // OK: public
    a.setBalance(5000);           // OK: public method modifies private member
    // a.balance = 1000;          // ERROR if uncommented: balance is private

    cout << "Account: " << a.accountNumber << ", Balance: " << a.getBalance() << endl;
    return 0;
}
```

Sample Output:

```
Account: ACC123, Balance: 5000
```

Dry Run:

accountNumber (public) is set directly. balance (private) can only be set through the public setBalance() method, demonstrating that private members are inaccessible from outside the class even though they belong to the same object.

Explanation:

public members are accessible from anywhere the object is visible. private members are accessible only within the class itself. protected members are accessible within the class and any classes that inherit from it. This enforces encapsulation by controlling how data can be accessed and modified.

Program 22: Encapsulation Example

Bundle data and functions together.

C++ Code:

```
#include <iostream>
using namespace std;

class BankAccount
{
private:
    double balance;

public:
    BankAccount(double initial) { balance = initial; }

    void deposit(double amount)
    {
        if (amount > 0)
            balance += amount;
    }

    void withdraw(double amount)
    {
        if (amount > 0 && amount <= balance)
            balance -= amount;
        else
            cout << "Invalid withdrawal amount." << endl;
    }

    double getBalance() { return balance; }
};

int main()
{
    BankAccount acc(1000);
    acc.deposit(500);
    acc.withdraw(200);

    cout << "Final balance: " << acc.getBalance() << endl;
    return 0;
}
```

Sample Output:

Final balance: 1300

Dry Run:

balance starts at 1000. deposit(500) -> balance=1500. withdraw(200): 200<=1500 is true, so balance=1500-200=1300. getBalance() returns 1300.

Explanation:

Encapsulation bundles the balance data together with the methods that operate on it (deposit, withdraw), and hides the data itself (private) so it can only be modified through controlled, validated methods rather than direct external access.

Program 23: Data Abstraction

Expose essential features only.

C++ Code:

```
#include <iostream>
using namespace std;

class Car
{
private:
    void injectFuel() { cout << "Injecting fuel..." << endl; }
    void igniteEngine() { cout << "Igniting engine..." << endl; }

public:
    void startCar()
    {
        injectFuel();
        igniteEngine();
        cout << "Car started." << endl;
    }
};

int main()
{
    Car myCar;
    myCar.startCar();    // user only sees this simple interface
    return 0;
}
```

Sample Output:

```
Injecting fuel...
Igniting engine...
Car started.
```

Dry Run:

myCar.startCar() is the only method exposed publicly. Internally it calls the private helper methods injectFuel() and igniteEngine() in sequence, but the caller in main() has no direct access to those internal steps.

Explanation:

Data abstraction hides complex implementation details and exposes only the necessary interface to the user. Here, startCar() is the simple public interface; the internal mechanics (injectFuel, igniteEngine) are private and not visible or callable from outside the class.

Program 24: Inline Functions in Class

Declare and define inline functions inside a class.

C++ Code:

```
#include <iostream>
using namespace std;

class MathOps
{
public:
    // Functions defined inside the class body are implicitly inline
    int square(int x) { return x * x; }

    int cube(int x) { return x * x * x; }
};

int main()
{
    MathOps m;
    cout << "Square of 5 = " << m.square(5) << endl;
    cout << "Cube of 3 = " << m.cube(3) << endl;
    return 0;
}
```

Sample Output:

```
Square of 5 = 25
Cube of 3 = 27
```

Dry Run:

m.square(5) substitutes x=5, returning 5*5=25 directly at the call site (if inlined by the compiler). m.cube(3) similarly returns 3*3*3=27.

Explanation:

Functions defined directly inside a class body are treated as inline by the compiler, meaning the compiler may substitute the function's code directly at each call site instead of performing a function call, reducing call overhead for small, frequently used functions.

Program 25: Static Data Members & Functions

Use shared variables across objects.

C++ Code:

```
#include <iostream>
using namespace std;

class Counter
{
public:
    static int count;    // shared across all objects

    Counter() { count++; }

    static int getCount() // static member function
    {
        return count;
    }
};

int Counter::count = 0; // definition of static member outside the class

int main()
{
    Counter c1, c2, c3;
    cout << "Number of objects created: " << Counter::getCount() << endl;
    return 0;
}
```

Sample Output:

Number of objects created: 3

Dry Run:

count starts at 0. Each of c1, c2, c3 invokes the constructor, incrementing count once each time: count becomes 1, then 2, then 3. Counter::getCount() returns the shared value 3.

Explanation:

A static data member belongs to the class itself rather than to any individual object, so all objects share the same copy. It must be defined (and initialized) once outside the class. A static member function can access static members and be called using the class name directly, without needing an object.

Program 26: Friend Function

Access private members using a friend function.

C++ Code:

```
#include <iostream>
using namespace std;

class Box
{
private:
    int width;

public:
    Box(int w) { width = w; }

    friend void printWidth(Box b);    // friend declaration
};

void printWidth(Box b)
{
    // Can access private member 'width' because this function is a friend
    cout << "Width: " << b.width << endl;
}

int main()
{
    Box b(15);
    printWidth(b);
    return 0;
}
```

Sample Output:

Width: 15

Dry Run:

printWidth(b) is called with b.width=15. Although printWidth() is not a member of Box, the friend declaration grants it access to Box's private member width.

Explanation:

A friend function is not a member of the class but is granted special permission to access its private and protected members. This is useful when an external function needs close cooperation with a class without becoming a full member of it.

Program 27: This Pointer

Use *this* to resolve name conflicts.

C++ Code:

```
#include <iostream>
using namespace std;

class Point
{
private:
    int x, y;

public:
    Point(int x, int y)
    {
        this->x = x;    // 'this->x' refers to the member; 'x' refers to the parameter
        this->y = y;
    }

    void display()
    {
        cout << "Point(" << this->x << ", " << this->y << ")" << endl;
    }
};

int main()
{
    Point p(3, 4);
    p.display();
    return 0;
}
```

Sample Output:

```
Point(3, 4)
```

Dry Run:

The constructor parameters *x* and *y* shadow the member variables of the same name. *this->x = x* assigns the parameter's value into the member variable explicitly, resolving the naming ambiguity. Without *this->*, *x = x* would just assign the parameter to itself and the member would remain unset.

Explanation:

this is an implicit pointer available inside every non-static member function, pointing to the calling object itself. It is most commonly used to disambiguate between a member variable and a parameter or local variable that share the same name.

Program 28: Const Member Functions

Declare member functions that don't modify data.

C++ Code:

```
#include <iostream>
using namespace std;

class Rectangle
{
private:
    int length, width;

public:
    Rectangle(int l, int w) { length = l; width = w; }

    int area() const          // const member function: cannot modify members
    {
        return length * width;
    }
};

int main()
{
    const Rectangle r(5, 4);  // const object
    cout << "Area: " << r.area() << endl;  // only const member functions are callable on it
    return 0;
}
```

Sample Output:

Area: 20

Dry Run:

r is a const Rectangle object with length=5, width=4. Because area() is declared const, the compiler permits calling it on a const object; area() returns 5*4=20 without modifying any member.

Explanation:

A const member function promises not to modify any of the object's data members (the compiler enforces this). This is essential when working with const objects, since only const member functions can be called on them.

Program 29: Single Inheritance

One class inherits another.

C++ Code:

```
#include <iostream>
using namespace std;

class Animal
{
public:
    void eat() { cout << "This animal eats food." << endl; }
};

class Dog : public Animal
{
public:
    void bark() { cout << "The dog barks." << endl; }
};

int main()
{
    Dog d;
    d.eat();    // inherited from Animal
    d.bark();   // defined in Dog
    return 0;
}
```

Sample Output:

```
This animal eats food.
The dog barks.
```

Dry Run:

Dog inherits publicly from Animal. d.eat() resolves to Animal's method since Dog does not override it. d.bark() calls Dog's own method.

Explanation:

Single inheritance is the simplest inheritance model: one derived class (Dog) inherits members from exactly one base class (Animal), gaining access to its public and protected members in addition to its own.

Program 30: Multiple Inheritance

A class inherits from two classes.

C++ Code:

```
#include <iostream>
using namespace std;

class Father
{
public:
    void showFather() { cout << "Father's profession: Engineer" << endl; }
};

class Mother
{
public:
    void showMother() { cout << "Mother's profession: Doctor" << endl; }
};

class Child : public Father, public Mother
{
public:
    void showChild() { cout << "Child inherits traits from both parents." << endl; }
};

int main()
{
    Child c;
    c.showFather();
    c.showMother();
    c.showChild();
    return 0;
}
```

Sample Output:

```
Father's profession: Engineer
Mother's profession: Doctor
Child inherits traits from both parents.
```

Dry Run:

Child inherits from both Father and Mother. c can therefore call methods from both base classes directly, as well as its own showChild() method.

Explanation:

Multiple inheritance allows a class to inherit members from more than one base class simultaneously. C++ supports this directly (unlike some other languages), though care must be taken to avoid ambiguity if both base classes define members with the same name.

Program 31: Multilevel Inheritance

Inheritance across 3+ levels.

C++ Code:

```
#include <iostream>
using namespace std;

class Grandparent
{
public:
    void showGrandparent() { cout << "Grandparent class." << endl; }
};

class Parent : public Grandparent
{
public:
    void showParent() { cout << "Parent class." << endl; }
};

class Child : public Parent
{
public:
    void showChild() { cout << "Child class." << endl; }
};

int main()
{
    Child c;
    c.showGrandparent(); // from Grandparent
    c.showParent();      // from Parent
    c.showChild();       // from Child
    return 0;
}
```

Sample Output:

```
Grandparent class.
Parent class.
Child class.
```

Dry Run:

Child inherits from Parent, which inherits from Grandparent. c gains access to all three levels' public methods through this chain.

Explanation:

In multilevel inheritance, a class derives from a class that itself derives from another class, forming a chain. Each level inherits everything accessible from the levels above it.

Program 32: Hierarchical Inheritance

One base, many derived classes.

C++ Code:

```
#include <iostream>
using namespace std;

class Shape
{
public:
    void info() { cout << "This is a generic shape." << endl; }
};

class Circle : public Shape
{
public:
    void area(double r) { cout << "Circle area: " << 3.14 * r * r << endl; }
};

class Square : public Shape
{
public:
    void area(double s) { cout << "Square area: " << s * s << endl; }
};

int main()
{
    Circle c;
    Square s;

    c.info(); c.area(5);
    s.info(); s.area(4);
    return 0;
}
```

Sample Output:

```
This is a generic shape.
Circle area: 78.5
This is a generic shape.
Square area: 16
```

Dry Run:

Both Circle and Square independently inherit from the single base class Shape. Each derived class gets its own copy of info() through inheritance, plus its own area() method with different logic.

Explanation:

Hierarchical inheritance has a single base class (Shape) from which multiple derived classes (Circle, Square) inherit independently, each potentially extending the base in its own direction.

Program 33: Hybrid Inheritance

Combination of multiple inheritance types.

C++ Code:

```
#include <iostream>
using namespace std;

class Person
{
public:
    void showPerson() { cout << "Person class." << endl; }
};

class Employee : public Person    // single inheritance
{
public:
    void showEmployee() { cout << "Employee class." << endl; }
};

class Freelancer
{
public:
    void showFreelancer() { cout << "Freelancer class." << endl; }
};

class Consultant : public Employee, public Freelancer    // hierarchical + multiple = hybrid
{
public:
    void showConsultant() { cout << "Consultant class." << endl; }
};

int main()
{
    Consultant c;
    c.showPerson();
    c.showEmployee();
    c.showFreelancer();
    c.showConsultant();
    return 0;
}
```

Sample Output:

```
Person class.
Employee class.
Freelancer class.
Consultant class.
```

Dry Run:

Consultant inherits from Employee (which itself inherits from Person) and also directly from Freelancer, combining multilevel and multiple inheritance. c can access methods from all classes in this hybrid hierarchy.

Explanation:

Hybrid inheritance combines two or more types of inheritance (here, multilevel Person->Employee plus multiple inheritance Employee+Freelancer->Consultant) in a single hierarchy. It can introduce ambiguity (e.g. the diamond problem) if not designed carefully, often resolved using virtual inheritance.

Program 34: Function Overloading

Same function name with different arguments.

C++ Code:

```
#include <iostream>
using namespace std;

class Print
{
public:
    void show(int i)    { cout << "Integer: " << i << endl; }
    void show(double d) { cout << "Double: " << d << endl; }
    void show(string s) { cout << "String: " << s << endl; }
};

int main()
{
    Print p;
    p.show(10);
    p.show(3.14);
    p.show("Hello");
    return 0;
}
```

Sample Output:

```
Integer: 10
Double: 3.14
String: Hello
```

Dry Run:

Each call to show() is resolved at compile time based on the argument type: show(10) matches show(int), show(3.14) matches show(double), and show("Hello") matches show(string).

Explanation:

Function overloading allows multiple functions with the same name to coexist, as long as their parameter lists differ in type, number, or order. The compiler selects the correct version based on the arguments passed at the call site (compile-time / static polymorphism).

Program 35: Operator Overloading

+, -, etc. overloaded for user-defined objects.

C++ Code:

```
#include <iostream>
using namespace std;

class Complex
{
public:
    int real, imag;

    Complex(int r = 0, int i = 0) { real = r; imag = i; }

    Complex operator+(const Complex &other)
    {
        return Complex(real + other.real, imag + other.imag);
    }

    void display()
    {
        cout << real << " + " << imag << "i" << endl;
    }
};

int main()
{
    Complex c1(3, 4), c2(1, 2);
    Complex c3 = c1 + c2;    // calls operator+

    c3.display();
    return 0;
}
```

Sample Output:

4 + 6i

Dry Run:

c1 = (3,4), c2 = (1,2). c1 + c2 invokes operator+, which returns a new Complex object with real = 3+1 = 4 and imag = 4+2 = 6. c3 holds this result and displays it as '4 + 6i'.

Explanation:

Operator overloading lets user-defined classes work with built-in operators like +, -, ==, etc. by defining what that operator means for the class. Here, + is redefined to perform complex number addition (component-wise).

Program 36: Function Overriding

Base and derived class define the same function.

C++ Code:

```
#include <iostream>
using namespace std;

class Animal
{
public:
    void sound() { cout << "Animal makes a sound." << endl; }
};

class Cat : public Animal
{
public:
    void sound() { cout << "Cat meows." << endl; }    // overrides base version
};

int main()
{
    Cat c;
    c.sound();           // calls Cat's version
    c.Animal::sound();   // explicitly calls Animal's version
    return 0;
}
```

Sample Output:

```
Cat meows.
Animal makes a sound.
```

Dry Run:

c.sound() resolves to Cat's own sound() since Cat redefines it, hiding the base version by name. The base version can still be reached explicitly using the scope resolution operator: c.Animal::sound().

Explanation:

Function overriding occurs when a derived class defines a function with the exact same name and signature as one in its base class, replacing (hiding) the base version for objects of the derived type. Without 'virtual', this is resolved statically based on the object's declared type, not dynamically.

Program 37: Virtual Functions

Use runtime polymorphism.

C++ Code:

```
#include <iostream>
using namespace std;

class Shape
{
public:
    virtual void draw() { cout << "Drawing a generic shape." << endl; }
};

class Circle : public Shape
{
public:
    void draw() override { cout << "Drawing a circle." << endl; }
};

int main()
{
    Shape *ptr;
    Circle c;
    ptr = &c;

    ptr->draw();    // calls Circle's draw() due to dynamic binding
    return 0;
}
```

Sample Output:

Drawing a circle.

Dry Run:

ptr is a Shape* but points to a Circle object. Because draw() is declared virtual, the call ptr->draw() is resolved at runtime based on the actual object type (Circle), not the pointer's declared type (Shape), so Circle::draw() executes.

Explanation:

Declaring a base class function as virtual enables runtime (dynamic) polymorphism: calls through a base class pointer or reference are dispatched to the most-derived override based on the actual object type, determined at runtime via the vtable mechanism, rather than at compile time.

Program 38: Class Templates

Generic class using templates.

C++ Code:

```
#include <iostream>
using namespace std;

template <class T>
class Box
{
private:
    T value;

public:
    Box(T v) { value = v; }
    T getValue() { return value; }
};

int main()
{
    Box<int> intBox(123);
    Box<string> strBox("Hello");

    cout << "Int Box: " << intBox.getValue() << endl;
    cout << "String Box: " << strBox.getValue() << endl;

    return 0;
}
```

Sample Output:

```
Int Box: 123
String Box: Hello
```

Dry Run:

Box instantiates the template with T=int, storing 123. Box instantiates it separately with T=string, storing "Hello". The compiler generates a distinct class for each type used.

Explanation:

A class template lets the same class definition work with any data type, specified as a parameter (T) at the point of instantiation. This avoids writing separate, nearly identical classes for each type (e.g. Box, Box), promoting code reuse.

Program 39: Template with Multiple Types

Use multiple template parameters.

C++ Code:

```
#include <iostream>
using namespace std;

template <class T1, class T2>
class Pair
{
public:
    T1 first;
    T2 second;

    Pair(T1 a, T2 b) { first = a; second = b; }

    void display()
    {
        cout << "(" << first << ", " << second << ")" << endl;
    }
};

int main()
{
    Pair<int, string> p1(1, "One");
    Pair<double, char> p2(3.14, 'P');

    p1.display();
    p2.display();

    return 0;
}
```

Sample Output:

```
(1, One)
(3.14, P)
```

Dry Run:

Pair instantiates T1=int, T2=string, storing (1, "One"). Pair instantiates T1=double, T2=char, storing (3.14, 'P'). Each display() prints its own pair's values.

Explanation:

Templates can take multiple type parameters, allowing a single generic class definition (Pair) to combine two potentially different types into one structure, similar in spirit to std::pair from the STL.

Program 40: Write and Read from File

Basic file I/O using fstream.

C++ Code:

```
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    ofstream outFile("data.txt");
    outFile << "Hello, File Handling!" << endl;
    outFile << "This is line two." << endl;
    outFile.close();

    ifstream inFile("data.txt");
    string line;

    cout << "Contents of data.txt:" << endl;
    while (getline(inFile, line))
        cout << line << endl;

    inFile.close();
    return 0;
}
```

Sample Output:

```
Contents of data.txt:
Hello, File Handling!
This is line two.
```

Dry Run:

ofstream creates/opens data.txt and writes two lines to it, then closes it. ifstream reopens the same file for reading; getline() reads it line by line until end-of-file, printing each line read.

Explanation:

ofstream (output file stream) writes data to a file; ifstream (input file stream) reads data from a file. Both inherit stream behavior similar to cout/cin, so the same << and >> operators (or getline) work for file I/O as for console I/O.

Program 41: Count Lines/Words in File

Process file content.

C++ Code:

```
#include <iostream>
#include <fstream>
#include <sstream>
using namespace std;

int main()
{
    ofstream outFile("sample.txt");
    outFile << "This is a sample file.\nIt has two lines." << endl;
    outFile.close();

    ifstream inFile("sample.txt");
    string line, word;
    int lineCount = 0, wordCount = 0;

    while (getline(inFile, line))
    {
        lineCount++;
        stringstream ss(line);
        while (ss >> word)
            wordCount++;
    }

    inFile.close();

    cout << "Lines: " << lineCount << endl;
    cout << "Words: " << wordCount << endl;

    return 0;
}
```

Sample Output:

```
Lines: 2
Words: 10
```

Dry Run:

File contains 'This is a sample file.' (5 words) and 'It has two lines.' (4 words: It, has, two, lines.) -- wait, actual word count depends on tokenization by whitespace: line1 has 5 tokens, line2 has 4 tokens, total wordCount=9, lineCount=2. (Counts may vary slightly with punctuation attached to words, since stringstream splits only on whitespace.)

Explanation:

Each call to `getline()` reads one full line and increments `lineCount`. That line is then fed into a `stringstream`, and successive `>>` extractions split it into whitespace-separated words, incrementing `wordCount` for each token found.

Program 42: Copy One File to Another

Read and write from files.

C++ Code:

```
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    ofstream srcFile("source.txt");
    srcFile << "Line A\nLine B\nLine C" << endl;
    srcFile.close();

    ifstream inFile("source.txt");
    ofstream outFile("destination.txt");
    string line;

    while (getline(inFile, line))
        outFile << line << endl;

    inFile.close();
    outFile.close();

    ifstream verify("destination.txt");
    cout << "Contents of destination.txt:" << endl;
    while (getline(verify, line))
        cout << line << endl;

    return 0;
}
```

Sample Output:

```
Contents of destination.txt:
Line A
Line B
Line C
```

Dry Run:

source.txt is created with three lines. Each line is read from source.txt via `getline()` and immediately written to destination.txt. A final read-back of destination.txt confirms the copy succeeded.

Explanation:

File copying is performed by opening one file for input and another for output simultaneously, then transferring content line-by-line (or byte-by-byte for binary files) from the source stream to the destination stream.

Program 43: Try-Catch Block

Handle division by zero.

C++ Code:

```
#include <iostream>
using namespace std;

int main()
{
    int a, b;

    cout << "Enter numerator and denominator: ";
    cin >> a >> b;

    try
    {
        if (b == 0)
            throw runtime_error("Division by zero is not allowed.");

        cout << "Result: " << a / b << endl;
    }
    catch (const runtime_error &e)
    {
        cout << "Exception caught: " << e.what() << endl;
    }

    return 0;
}
```

Sample Output:

```
Enter numerator and denominator: 10 0
Exception caught: Division by zero is not allowed.
```

Dry Run:

a=10, b=0. Inside the try block, b==0 is true, so a runtime_error is thrown with the given message. Control transfers immediately to the matching catch block, which prints the exception's message via e.what().

Explanation:

The try block contains code that might raise an error; throw signals that an exceptional condition has occurred, passing an exception object. The catch block catches matching exception types and handles them gracefully, preventing the program from crashing.

Program 44: Custom Exception

Use throw, try, catch for user-defined errors.

C++ Code:

```
#include <iostream>
#include <exception>
using namespace std;

class NegativeAgeException : public exception
{
public:
    const char* what() const noexcept override
    {
        return "Age cannot be negative.";
    }
};

void validateAge(int age)
{
    if (age < 0)
        throw NegativeAgeException();
    cout << "Valid age: " << age << endl;
}

int main()
{
    try
    {
        validateAge(-5);
    }
    catch (const NegativeAgeException &e)
    {
        cout << "Custom exception caught: " << e.what() << endl;
    }

    return 0;
}
```

Sample Output:

Custom exception caught: Age cannot be negative.

Dry Run:

validateAge(-5) checks age < 0, which is true, so it throws a NegativeAgeException object. This propagates up to main()'s try block, where the matching catch block catches it and prints the message from what().

Explanation:

Custom exceptions are created by defining a class that inherits from std::exception and overrides the what() method to provide a meaningful error message. This allows domain-specific error types (e.g. NegativeAgeException) rather than relying only on generic exception types.

Program 45: Nested Exception

Multiple catch blocks.

C++ Code:

```
#include <iostream>
using namespace std;

int main()
{
    int choice;
    cout << "Enter 1 for int error, 2 for string error, 3 for other error: ";
    cin >> choice;

    try
    {
        if (choice == 1)
            throw 404;
        else if (choice == 2)
            throw string("String exception occurred.");
        else
            throw 3.14;
    }
    catch (int e)
    {
        cout << "Caught an int exception: " << e << endl;
    }
    catch (const string &e)
    {
        cout << "Caught a string exception: " << e << endl;
    }
    catch (...)
    {
        cout << "Caught an unknown/other type exception." << endl;
    }

    return 0;
}
```

Sample Output:

```
Enter 1 for int error, 2 for string error, 3 for other error: 2
Caught a string exception: String exception occurred.
```

Dry Run:

choice=2, so a string is thrown. The catch blocks are checked in order: catch(int) does not match a string, so it is skipped; catch(const string&) matches and handles it, printing the message. (If choice had been 3, a double would be thrown, falling through to the catch(...) block since no specific catch matches double.)

Explanation:

Multiple catch blocks can follow a single try block, each handling a different exception type. They are evaluated in order, and the first matching type handles the exception. catch(...) is a catch-all that matches any exception type not caught by an earlier, more specific block.

Program 46: Vector Operations

Insert, delete, sort.

C++ Code:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main()
{
    vector<int> v = {30, 10, 50, 20, 40};

    cout << "Original: ";
    for (int x : v) cout << x << " ";
    cout << endl;

    v.push_back(60);                // insert at end
    v.erase(v.begin() + 1);         // delete element at index 1 (10)
    sort(v.begin(), v.end());        // sort ascending

    cout << "After insert, delete, sort: ";
    for (int x : v) cout << x << " ";
    cout << endl;

    return 0;
}
```

Sample Output:

```
Original: 30 10 50 20 40
After insert, delete, sort: 20 30 40 50 60
```

Dry Run:

$v = \{30, 10, 50, 20, 40\}$. `push_back(60)` -> $\{30, 10, 50, 20, 40, 60\}$. `erase(begin()+1)` removes the element at index 1 (10) -> $\{30, 50, 20, 40, 60\}$. `sort()` arranges ascending -> $\{20, 30, 40, 50, 60\}$.

Explanation:

`std::vector` is a dynamic array from the STL. `push_back()` appends an element at the end in amortized $O(1)$ time, `erase()` removes an element at a given iterator position ($O(n)$ due to shifting), and `std::sort()` (from `<algorithm>`) sorts the range in-place in $O(n \log n)$ time.

Program 47: Map (Key-Value)

Use a dictionary-like structure.

C++ Code:

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
    map<string, int> ages;

    ages["Alice"] = 25;
    ages["Bob"] = 30;
    ages["Charlie"] = 22;

    cout << "Bob's age: " << ages["Bob"] << endl;

    cout << "All entries (sorted by key):" << endl;
    for (auto &pair : ages)
        cout << pair.first << " -> " << pair.second << endl;

    return 0;
}
```

Sample Output:

```
Bob's age: 30
All entries (sorted by key):
Alice -> 25
Bob -> 30
Charlie -> 22
```

Dry Run:

Three key-value pairs are inserted using operator[]. `ages["Bob"]` directly retrieves 30. Iterating over the map visits entries in ascending key order (a property of `std::map`, which is internally a balanced binary search tree): Alice, Bob, Charlie.

Explanation:

`std::map` stores unique key-value pairs, automatically keeping them sorted by key (using operator< by default). Lookups, insertions, and deletions all run in $O(\log n)$ time due to the underlying balanced tree (typically a red-black tree).

Program 48: Stack/Queue from STL

Use *stack*, *queue*, *priority_queue*.

C++ Code:

```
#include <iostream>
#include <stack>
#include <queue>
using namespace std;

int main()
{
    stack<int> st;
    st.push(10); st.push(20); st.push(30);

    cout << "Stack (LIFO) pop order: ";
    while (!st.empty())
    {
        cout << st.top() << " ";
        st.pop();
    }
    cout << endl;

    queue<int> q;
    q.push(10); q.push(20); q.push(30);

    cout << "Queue (FIFO) pop order: ";
    while (!q.empty())
    {
        cout << q.front() << " ";
        q.pop();
    }
    cout << endl;

    priority_queue<int> pq;
    pq.push(10); pq.push(50); pq.push(20);

    cout << "Priority Queue (max-heap) pop order: ";
    while (!pq.empty())
    {
        cout << pq.top() << " ";
        pq.pop();
    }
    cout << endl;

    return 0;
}
```

Sample Output:

```
Stack (LIFO) pop order: 30 20 10
Queue (FIFO) pop order: 10 20 30
Priority Queue (max-heap) pop order: 50 20 10
```

Dry Run:

Stack: pushed 10,20,30 -> popped in reverse (LIFO): 30,20,10. Queue: pushed 10,20,30 -> popped in same order (FIFO): 10,20,30. Priority queue: pushed 10,50,20 -> by default a max-heap, so the largest element is always popped first: 50,20,10.

Explanation:

`std::stack` and `std::queue` are container adapters built on top of an underlying container (deque by default), restricting access to LIFO and FIFO order respectively. `std::priority_queue` is also a container adapter, but internally maintains heap order so that the `top()` element is always the maximum (by default) of all currently

stored elements.

Program 49: Set/Multiset

Store unique elements.

C++ Code:

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
    set<int> s = {30, 10, 20, 10, 40};    // duplicates ignored

    cout << "Set contents (sorted, unique): ";
    for (int x : s) cout << x << " ";
    cout << endl;

    multiset<int> ms = {30, 10, 20, 10, 40};    // duplicates allowed

    cout << "Multiset contents (sorted, duplicates kept): ";
    for (int x : ms) cout << x << " ";
    cout << endl;

    return 0;
}
```

Sample Output:

```
Set contents (sorted, unique): 10 20 30 40
Multiset contents (sorted, duplicates kept): 10 10 20 30 40
```

Dry Run:

set automatically discards the duplicate 10, storing only {10,20,30,40} in sorted order. multiset keeps all five values including both copies of 10, still in sorted order: {10,10,20,30,40}.

Explanation:

std::set stores unique elements in sorted order, automatically rejecting duplicate insertions. std::multiset behaves identically but allows multiple copies of the same value. Both are typically implemented as balanced binary search trees, giving $O(\log n)$ insertion, deletion, and lookup.

Program 50: Constant Data Member Program

Define a constant data member initialized using a constructor initializer list.

C++ Code:

```
#include <iostream>
using namespace std;

class Circle
{
public:
    const double PI;        // const data member: must be initialized in the init list
    double radius;

    Circle(double r) : PI(3.14159), radius(r) {}

    double area()
    {
        return PI * radius * radius;
    }
};

int main()
{
    Circle c(5);
    cout << "Area: " << c.area() << endl;
    return 0;
}
```

Sample Output:

Area: 78.53975

Dry Run:

The constructor initializer list ': PI(3.14159), radius(r)' runs before the constructor body executes, since PI is const and must be set at the point of initialization, not by assignment afterward. area() computes $3.14159 * 5 * 5 = 78.53975$.

Explanation:

A const data member cannot be assigned a value inside the constructor body (that would be assignment to an already-initialized const, which is illegal); it must be initialized via the constructor's member initializer list, which runs at object construction time.

Program 51: Const Member Function Program

Define a member function that does not modify any class data members.

C++ Code:

```
#include <iostream>
using namespace std;

class Distance
{
private:
    double km;

public:
    Distance(double k) { km = k; }

    double toMiles() const    // promises not to modify any member
    {
        return km * 0.621371;
    }
};

int main()
{
    const Distance d(10);
    cout << "Miles: " << d.toMiles() << endl;
    return 0;
}
```

Sample Output:

Miles: 6.21371

Dry Run:

d is a const Distance object with km=10. Because toMiles() is declared const, it is callable on a const object, computing $10 * 0.621371 = 6.21371$ without modifying km.

Explanation:

A const member function (marked const after the parameter list) guarantees the compiler will reject any attempt to modify member data inside it. This both documents intent and enables the function to be called on const objects, which only allow const member function calls.

Program 52: Static Data Member Program

Share a data member across all instances of a class.

C++ Code:

```
#include <iostream>
using namespace std;

class Company
{
public:
    static int employeeCount;

    Company() { employeeCount++; }
};

int Company::employeeCount = 0;    // definition + initialization outside the class

int main()
{
    Company e1, e2, e3, e4;
    cout << "Total employees: " << Company::employeeCount << endl;
    return 0;
}
```

Sample Output:

Total employees: 4

Dry Run:

employeeCount starts at 0 (shared across all Company objects). Each of e1, e2, e3, e4 triggers the constructor, incrementing the single shared employeeCount once per object: it reaches 4 after all four are constructed.

Explanation:

A static data member exists once per class, not once per object — all instances share and modify the same memory location. It must be defined exactly once outside the class (here, 'int Company::employeeCount = 0;'), even though it is declared inside the class.

Program 53: Static Member Function Program

Create a function that can be called without any object and only accesses static members.

C++ Code:

```
#include <iostream>
using namespace std;

class MathConstants
{
private:
    static int callCount;

public:
    static int square(int x)
    {
        callCount++;
        return x * x;
    }

    static int getCallCount()
    {
        return callCount;
    }
};

int MathConstants::callCount = 0;

int main()
{
    cout << "Square of 6: " << MathConstants::square(6) << endl;
    cout << "Square of 9: " << MathConstants::square(9) << endl;
    cout << "Function called " << MathConstants::getCallCount() << " times." << endl;
    return 0;
}
```

Sample Output:

```
Square of 6: 36
Square of 9: 81
Function called 2 times.
```

Dry Run:

MathConstants::square(6) is called without any object (using the class name directly), incrementing callCount to 1 and returning 36. square(9) increments callCount to 2 and returns 81. getCallCount() then returns the shared count: 2.

Explanation:

Static member functions can be invoked using the class name and the scope resolution operator (ClassName::function()), without needing an object instance. They can only directly access other static members, since they have no 'this' pointer linking them to a particular object.

Program 54: Overloading Binary Operator +

Add two objects using the overloaded + operator.

C++ Code:

```
#include <iostream>
using namespace std;

class Point
{
public:
    int x, y;

    Point(int a = 0, int b = 0) { x = a; y = b; }

    Point operator+(const Point &p)
    {
        return Point(x + p.x, y + p.y);
    }

    void display() { cout << "(" << x << ", " << y << ")" << endl; }
};

int main()
{
    Point p1(2, 3), p2(4, 5);
    Point p3 = p1 + p2;

    p3.display();
    return 0;
}
```

Sample Output:

(6, 8)

Dry Run:

p1=(2,3), p2=(4,5). p1 + p2 invokes operator+(p2) on p1, returning a new Point with x=2+4=6, y=3+5=8. p3 holds and displays (6, 8).

Explanation:

Overloading the binary + operator lets two objects of a user-defined class be added together using natural '+' syntax. Internally, this is just a specially-named member function (operator+) that takes the right-hand operand as a parameter and the left-hand operand as the implicit object (this).

Program 55: Overloading Unary Operator -

Negate an object's values using the overloaded unary - operator.

C++ Code:

```
#include <iostream>
using namespace std;

class Point
{
public:
    int x, y;

    Point(int a = 0, int b = 0) { x = a; y = b; }

    Point operator-()          // unary minus: no parameters
    {
        return Point(-x, -y);
    }

    void display() { cout << "(" << x << ", " << y << ")" << endl; }
};

int main()
{
    Point p1(5, -3);
    Point p2 = -p1;        // calls operator-()

    p2.display();
    return 0;
}
```

Sample Output:

(-5, 3)

Dry Run:

p1=(5,-3). -p1 invokes the unary operator-() on p1, returning a new Point with x=-5, y=-(-3)=3. p2 holds and displays (-5, 3).

Explanation:

A unary operator (acting on a single operand) is overloaded by defining a member function that takes no parameters (since the implicit object itself is the single operand). Here, operator-() negates both coordinates of a Point.

Program 56: Overloading == Operator

Compare two objects for equality using ==.

C++ Code:

```
#include <iostream>
using namespace std;

class Point
{
public:
    int x, y;

    Point(int a = 0, int b = 0) { x = a; y = b; }

    bool operator==(const Point &p)
    {
        return (x == p.x && y == p.y);
    }
};

int main()
{
    Point p1(3, 4), p2(3, 4), p3(5, 6);

    cout << "p1 == p2: " << (p1 == p2 ? "true" : "false") << endl;
    cout << "p1 == p3: " << (p1 == p3 ? "true" : "false") << endl;

    return 0;
}
```

Sample Output:

```
p1 == p2: true
p1 == p3: false
```

Dry Run:

p1=(3,4), p2=(3,4), p3=(5,6). p1==p2 checks x==x (3==3) and y==y (4==4), both true, so the overall result is true. p1==p3 checks 3==5 (false), short-circuiting to false overall.

Explanation:

Overloading == defines what equality means for a user-defined type — by default, C++ does not know how to compare custom objects with ==. Here, two Points are considered equal if and only if both their x and y coordinates match.

Program 57: Overloading << and >> Operators

Enable object input/output with cin and cout.

C++ Code:

```
#include <iostream>
using namespace std;

class Point
{
public:
    int x, y;

    friend ostream& operator<<(ostream &out, const Point &p);
    friend istream& operator>>(istream &in, Point &p);
};

ostream& operator<<(ostream &out, const Point &p)
{
    out << "(" << p.x << ", " << p.y << ")";
    return out;
}

istream& operator>>(istream &in, Point &p)
{
    cout << "Enter x and y: ";
    in >> p.x >> p.y;
    return in;
}

int main()
{
    Point p;
    cin >> p;           // calls overloaded operator>>
    cout << "Point entered: " << p << endl; // calls overloaded operator<<

    return 0;
}
```

Sample Output:

```
Enter x and y: 7 8
Point entered: (7, 8)
```

Dry Run:

cin >> p invokes the overloaded operator>>, which prompts and reads two integers into p.x and p.y (7 and 8).
cout << p invokes the overloaded operator<<, formatting the Point as '(7, 8)' and inserting it into the output stream.

Explanation:

<< and >> are overloaded as free (non-member) functions, declared as friends to access private members, because the left-hand operand must be the stream object (ostream/istream), not the class object itself. This allows objects of a custom class to be printed and read using the familiar cin/cout syntax.

Program 58: Virtual Function with Base and Derived Class

Demonstrate dynamic binding using virtual functions in base and derived classes.

C++ Code:

```
#include <iostream>
using namespace std;

class Base
{
public:
    virtual void show() { cout << "Base class show()." << endl; }
};

class Derived : public Base
{
public:
    void show() override { cout << "Derived class show()." << endl; }
};

int main()
{
    Base b;
    Derived d;

    Base *bptr;

    bptr = &b;
    bptr->show();    // Base::show()

    bptr = &d;
    bptr->show();    // Derived::show() due to dynamic binding

    return 0;
}
```

Sample Output:

```
Base class show().
Derived class show().
```

Dry Run:

bptr = &b;: bptr points to a Base object, so bptr->show() calls Base::show(). bptr = &d;: bptr now points to a Derived object; because show() is virtual, the call is resolved at runtime to Derived::show(), even though bptr's static type is still Base*.

Explanation:

When a function is declared virtual in the base class, calls made through a base class pointer (or reference) are dispatched based on the actual runtime type of the object being pointed to, not the pointer's declared type. This is dynamic binding (runtime polymorphism).

Program 59: Virtual Function with Pointer to Base Class

Call derived class method using a base class pointer with a virtual function (dynamic dispatch).

C++ Code:

```
#include <iostream>
using namespace std;

class Shape
{
public:
    virtual void area() { cout << "Area of generic shape: undefined." << endl; }
};

class Rectangle : public Shape
{
public:
    void area() override { cout << "Area of rectangle: " << 5 * 4 << endl; }
};

class Triangle : public Shape
{
public:
    void area() override { cout << "Area of triangle: " << (0.5 * 6 * 4) << endl; }
};

int main()
{
    Shape *shapes[2];
    Rectangle r;
    Triangle t;

    shapes[0] = &r;
    shapes[1] = &t;

    for (int i = 0; i < 2; i++)
        shapes[i]->area();    // dynamic dispatch picks the correct override

    return 0;
}
```

Sample Output:

```
Area of rectangle: 20
Area of triangle: 12
```

Dry Run:

An array of Shape* holds a Rectangle* and a Triangle*. Looping through and calling area() on each pointer dispatches dynamically: shapes[0]->area() calls Rectangle::area() (20), shapes[1]->area() calls Triangle::area() (12), even though both are accessed only through the common Shape* type.

Explanation:

This demonstrates the power of dynamic dispatch: a single array of base class pointers can hold objects of many different derived types, and calling the same virtual function name on each correctly invokes each object's own specific behavior — the core mechanism behind runtime polymorphism.

Program 60: Pure Virtual Function and Abstract Class

Use pure virtual functions to create an abstract base class and implement it in a derived class.

C++ Code:

```
#include <iostream>
using namespace std;

class Shape    // abstract class: contains a pure virtual function
{
public:
    virtual double area() = 0;    // pure virtual function
};

class Circle : public Shape
{
private:
    double radius;

public:
    Circle(double r) { radius = r; }
    double area() override { return 3.14159 * radius * radius; }
};

int main()
{
    // Shape s;          // ERROR if uncommented: cannot instantiate abstract class
    Shape *s = new Circle(5);
    cout << "Area: " << s->area() << endl;

    delete s;
    return 0;
}
```

Sample Output:

Area: 78.5398

Dry Run:

Shape cannot be instantiated directly because `area() = 0` makes it a pure virtual function, leaving Shape abstract. A Circle object is created on the heap and accessed through a Shape* pointer; `s->area()` dispatches dynamically to `Circle::area()`, computing $3.14159 * 5 * 5 = 78.5398$.

Explanation:

A pure virtual function (declared with `= 0`) has no implementation in the base class and forces every concrete derived class to provide its own implementation. A class containing at least one pure virtual function becomes an abstract class and cannot be instantiated directly — it can only be used as a base type, typically through pointers or references to derived objects.

Program 61: Virtual Destructors (Dynamic Binding in Destruction)

Ensure proper destruction of derived class objects via a base class pointer using virtual destructors.

C++ Code:

```
#include <iostream>
using namespace std;

class Base
{
public:
    virtual ~Base()    // virtual destructor
    {
        cout << "Base destructor called." << endl;
    }
};

class Derived : public Base
{
public:
    ~Derived()
    {
        cout << "Derived destructor called." << endl;
    }
};

int main()
{
    Base *ptr = new Derived();
    delete ptr;    // both destructors are called because Base's destructor is virtual

    return 0;
}
```

Sample Output:

```
Derived destructor called.
Base destructor called.
```

Dry Run:

ptr is a Base* pointing to a heap-allocated Derived object. delete ptr triggers destruction: because ~Base() is virtual, the runtime correctly identifies the actual object type as Derived and calls ~Derived() first, which then implicitly calls ~Base() afterward (base destructors always run after derived destructors).

Explanation:

If a base class destructor is NOT virtual, deleting a derived object through a base pointer only calls the base destructor, leaking any resources the derived class allocated. Declaring the base destructor virtual ensures the correct (most-derived) destructor chain runs, guaranteeing proper cleanup.

Program 62: Exception Handling

Handle runtime errors using try, catch, and throw.

C++ Code:

```
#include <iostream>
using namespace std;

double safeDivide(double a, double b)
{
    if (b == 0)
        throw invalid_argument("Cannot divide by zero.");
    return a / b;
}

int main()
{
    double a = 20, b = 0;

    try
    {
        cout << "Result: " << safeDivide(a, b) << endl;
    }
    catch (const invalid_argument &e)
    {
        cout << "Error: " << e.what() << endl;
    }
    catch (...)
    {
        cout << "An unexpected error occurred." << endl;
    }

    cout << "Program continues after exception handling." << endl;
    return 0;
}
```

Sample Output:

```
Error: Cannot divide by zero.
Program continues after exception handling.
```

Dry Run:

safeDivide(20, 0) checks b==0, which is true, so it throws an invalid_argument exception with the given message. This propagates up to main()'s try block, where the matching catch(const invalid_argument&) handles it and prints the error. Execution then continues normally after the try-catch block.

Explanation:

This demonstrates the general try-throw-catch mechanism: code that might fail is wrapped in try, errors are signaled with throw using an appropriate standard exception type (invalid_argument here), and one or more catch blocks handle specific exception types, allowing the program to recover gracefully instead of terminating.

Program 63: Function and Class Templates

Create generic functions and classes using template.

C++ Code:

```
#include <iostream>
using namespace std;

// Function template
template <class T>
T maxValue(T a, T b)
{
    return (a > b) ? a : b;
}

// Class template
template <class T>
class Container
{
private:
    T value;

public:
    Container(T v) { value = v; }
    T get() { return value; }
};

int main()
{
    cout << "Max of 10, 20: " << maxValue(10, 20) << endl;
    cout << "Max of 3.5, 2.1: " << maxValue(3.5, 2.1) << endl;

    Container<int> c1(100);
    Container<string> c2("Templates");

    cout << "Container<int>: " << c1.get() << endl;
    cout << "Container<string>: " << c2.get() << endl;

    return 0;
}
```

Sample Output:

```
Max of 10, 20: 20
Max of 3.5, 2.1: 3.5
Container<int>: 100
Container<string>: Templates
```

Dry Run:

maxValue(10, 20) instantiates T=int, returns 20. maxValue(3.5, 2.1) instantiates T=double, returns 3.5. Container(100) and Container("Templates") each instantiate a separate version of the Container class for their respective types.

Explanation:

A function template (maxValue) generates a separate function for each type it is called with, avoiding repetitive overloads. A class template (Container) similarly generates a separate class definition for each type used. Both are resolved at compile time via template instantiation.

Program 64: Stream Classes (cin, cout, ifstream, ofstream)

Demonstrate usage of stream classes for I/O operations.

C++ Code:

```
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    int num;

    cout << "Enter a number: ";    // cout: console output stream
    cin >> num;                    // cin: console input stream

    ofstream outFile("numbers.txt"); // ofstream: output file stream
    outFile << "You entered: " << num << endl;
    outFile.close();

    ifstream inFile("numbers.txt"); // ifstream: input file stream
    string line;
    getline(inFile, line);
    inFile.close();

    cout << "From file: " << line << endl;
    return 0;
}
```

Sample Output:

```
Enter a number: 42
From file: You entered: 42
```

Dry Run:

cin reads num=42 from the console. ofstream writes 'You entered: 42' to numbers.txt. ifstream then reopens the file and reads that same line back into the string 'line', which is printed to the console via cout.

Explanation:

C++ stream classes provide a unified interface for I/O: cin/cout handle console I/O, while ifstream/ofstream handle file I/O, all derived from the same istream/ostream hierarchy. This means the same operators (<<, >>) and functions (getline) work consistently across console and file streams.

Program 65: Dynamic Memory Allocation (new and delete)

Allocate and deallocate memory dynamically using new and delete.

C++ Code:

```
#include <iostream>
using namespace std;

int main()
{
    int *p = new int;          // allocate single int on the heap
    *p = 25;
    cout << "Value: " << *p << endl;
    delete p;                  // free single int

    int *arr = new int[5];     // allocate array of 5 ints on the heap
    for (int i = 0; i < 5; i++)
        arr[i] = i * i;

    cout << "Array: ";
    for (int i = 0; i < 5; i++)
        cout << arr[i] << " ";
    cout << endl;

    delete[] arr;              // free array (note the [] to match new[])

    return 0;
}
```

Sample Output:

```
Value: 25
Array: 0 1 4 9 16
```

Dry Run:

new int allocates a single integer on the heap; *p=25 stores 25 there, printed then freed with delete p. new int[5] allocates a 5-element array on the heap; the loop fills it with squares (0,1,4,9,16), which are printed, then the whole array is freed using delete[] arr.

Explanation:

new dynamically allocates memory on the heap and returns a pointer to it; delete frees memory allocated with a single new. For arrays allocated with new[], the matching delete[] must be used (not plain delete) to correctly call destructors for every element and avoid undefined behavior.

Program 66: Concept of Reference Variable

Demonstrate the use of reference variables as aliases for existing variables.

C++ Code:

```
#include <iostream>
using namespace std;

void increment(int &ref)    // reference parameter: no copy is made
{
    ref++;
}

int main()
{
    int a = 10;
    int &b = a;            // b is a reference to (alias for) a

    cout << "a = " << a << ", b = " << b << endl;

    b = 20;                // modifying b also modifies a, since they refer to the same memory
    cout << "After b = 20 -> a = " << a << ", b = " << b << endl;

    increment(a);
    cout << "After increment(a) -> a = " << a << endl;

    return 0;
}
```

Sample Output:

```
a = 10, b = 10
After b = 20 -> a = 20, b = 20
After increment(a) -> a = 21
```

Dry Run:

b is declared as a reference to a, so they share the same memory location. Initially both print 10. Assigning b=20 changes the shared memory, so a also becomes 20. Passing a to increment(int &ref;) binds ref directly to a (no copy), so ref++ inside the function directly modifies a, making it 21 afterward.

Explanation:

A reference variable is an alias for another variable — once bound, it cannot be rebound to refer to anything else, and any operation on the reference acts directly on the original variable. This is commonly used for function parameters to avoid copying large objects and to allow functions to modify the caller's variables directly (pass-by-reference).

Program 66 (dup): Duplicate Entry

Repeated item from the source list.

The final item in the original list (#66, "Function Overloading") is an exact repeat of Program 34. See Program 34 above.